

OBJEKTER OG KLASSER

Objektorienteret programmering

OBJEKTORIENTERET PROGRAMMERING

Eksempel

Kondital kan estimeres ved en Coopers test, hvor personen tester, hvor langt han kan løbe i 12 minutter. Ud fra den tilbagelagte distance kan man ved hjælp af en formel beregne et estimat for konditallet

OBJEKTORIENTERET PROGRAMMERING

Eksempel

Kondital kan estimeres ved en Coopers test, hvor personen tester, hvor langt han kan løbe i 12 minutter. Ud fra den tilbagelagte distance kan man ved hjælp af en formel beregne et estimat for konditallet

```
navn = input('Indlæs dit navn ')\ndistance = int(input('Indlæs distancen i meter '))\nkondital = round((distance - 505)/45, 1)\nprint(f'{navn} - dit kondital er {kondital}')
```

```
Indlæs dit navn Peter\nIndlæs distancen i meter 3000\nPeter - dit kondital er 55.4
```

OBJEKTORIENTERET PROGRAMMERING

Eksempel

Kondital kan estimeres ved en Coopers test, hvor personen tester, hvor langt han kan løbe i 12 minutter. Ud fra den tilbagelagte distance kan man ved hjælp af en formel beregne et estimat for konditallet

```
navn = input('Indlæs dit navn ')\ndistance = int(input('Indlæs distancen i meter '))\nkondital = round((distance - 505)/45, 1)\nprint(f'{navn} - dit kondital er {kondital}')
```

```
Indlæs dit navn Peter\nIndlæs distancen i meter 3000\nPeter - dit kondital er 55.4
```

Forklar i detaljer, hvad der sker i de fire sætninger

OBJEKTORIENTERET PROGRAMMERING

Eksempel

```
navn = input('Indlæs dit navn ')\ndistance = int(input('Indlæs distancen i meter '))\nkondital = round((distance - 505)/45, 1)\nprint(f'{navn} - dit kondital er {kondital}')
```

OBJEKTORIENTERET PROGRAMMERING

Eksempel

```
navn = input('Indlæs dit navn ')\ndistance = int(input('Indlæs distancen i meter '))\nkondital = round((distance - 505)/45, 1)\nprint(f'{navn} - dit kondital er {kondital}')
```

```
class Person():\n    def __init__(self, navn):\n        self.navn = navn\n        self.distance = 0\n        self.kondital = 0\n\n    def test(self):\n        self.distance = int(input('Indlæs distancen i meter '))\n\n    def beregnKondital(self):\n        self.kondital = round((self.distance - 505)/45, 1)\n\n    def status(self):\n        return f'{self.navn} - dit kondital er {self.kondital}'
```

OBJEKTORIENTERET PROGRAMMERING

Eksempel

```
class Person():
    def __init__(self, navn):
        self.navn = navn
        self.distance = 0
        self.kondital = 0

    def test(self):
        self.distance = int(input('Indlæs distancen i meter '))

    def beregnKondital(self):
        self.kondital = round((self.distance - 505)/45, 1)

    def status(self):
        return f'{self.navn} - dit kondital er {self.kondital}'
```

```
navn = input('Indlæs dit navn ')
distance = int(input('Indlæs distancen i meter '))
kondital = round((distance - 505)/45, 1)
print(f'{navn} - dit kondital er {kondital}')
```

```
p = Person('Peter')
p.test()
p.beregnKondital()
p.status()
```

OBJEKTORIENTERET PROGRAMMERING

Eksempel

```
navn = input('Indlæs dit navn ')\ndistance = int(input('Indlæs distancen i meter '))\nkondital = round((distance - 505)/45, 1)\nprint(f'{navn} - dit kondital er {kondital}')
```

data

```
class Person():\n    def __init__(self, navn):\n        self.navn = navn\n        self.distance = 0\n        self.kondital = 0
```

```
p = Person('Peter')\np.test()\np.beregnKondital()\np.status()
```

```
def test(self):\n    self.distance = int(input('Indlæs distancen i meter '))\n\ndef beregnKondital(self):\n    self.kondital = round((self.distance - 505)/45, 1)\n\ndef status(self):\n    return f'{self.navn} - dit kondital er {self.kondital}'
```


OBJEKTORIENTERET PROGRAMMERING

Eksempel

```
navn = input('Indlæs dit navn ')\ndistance = int(input('Indlæs distancen i meter '))\nkondital = round((distance - 505)/45, 1)\nprint(f'{navn} - dit kondital er {kondital}')
```

class Person():

data

```
def __init__(self, navn):\n    self.navn = navn\n    self.distance = 0\n    self.kondital = 0
```

metoder

```
def test(self):\n    self.distance = int(input('Indlæs distancen i meter '))\n\ndef beregnKondital(self):\n    self.kondital = round((self.distance - 505)/45, 1)\n\ndef status(self):\n    return f'{self.navn} - dit kondital er {self.kondital}'
```

```
p = Person('Peter')\np.test()\np.beregnKondital()\np.status()
```

OBJEKTORIENTERET PROGRAMMERING

Et objekt

- modsvarer ting og begreber i den virkelige verden

OBJEKTORIENTERET PROGRAMMERING

Et objekt

- modsvarer ting og begreber i den virkelige verden
- kan modellere ting fra den virkelige verden

OBJEKTORIENTERET PROGRAMMERING

Et objekt

- modsvarer ting og begreber i den virkelige verden
- kan modellere ting fra den virkelige verden
- kan opfattes som en agent, der agerer med andre objekter (agenter)

OBJEKTORIENTERET PROGRAMMERING

Et objekt

- modsvarer ting og begreber i den virkelige verden
- kan modellere ting fra den virkelige verden
- kan opfattes som en agent, der agerer med andre objekter (agenter)
- har en tilstand bestemt ud fra data, som også kaldes attributter

OBJEKTORIENTERET PROGRAMMERING

Et objekt

- modsvarer ting og begreber i den virkelige verden
- kan modellere ting fra den virkelige verden
- kan opfattes som en agent, der agerer med andre objekter (agenter)
- har en tilstand bestemt ud fra data, som også kaldes attributter
- kan udføre handlinger, som kaldes metoder

OBJEKTORIENTERET PROGRAMMERING

Et objekt

- modsvarer ting og begreber i den virkelige verden
- kan modellere ting fra den virkelige verden
- kan opfattes som en agent, der agerer med andre objekter (agenter)
- har en tilstand bestemt ud fra data, som også kaldes attributter
- kan udføre handlinger, som kaldes metoder
- indkapsler sine data og sine metoder

OBJEKTORIENTERET PROGRAMMERING

Et objekt

- modsvarer ting og begreber i den virkelige verden
- kan modellere ting fra den virkelige verden
- kan opfattes som en agent, der agerer med andre objekter (agenter)
- har en tilstand bestemt ud fra data, som også kaldes attributter
- kan udføre handlinger, som kaldes metoder
- indkapsler sine data og sine metoder
- er konstrueret ud fra en skabelon (= en klasse)

OBJEKTORIENTERET PROGRAMMERING

Et objekt

- modsvarer ting og begreber i den virkelige verden
- kan modellere ting fra den virkelige verden
- kan opfattes som en agent, der agerer med andre objekter (agenter)
- har en tilstand bestemt ud fra data, som også kaldes attributter
- kan udføre handlinger, som kaldes metoder
- indkapsler sine data og sine metoder
- er konstrueret ud fra en skabelon (= en klasse)

En klasse

- er en samling af samme slags objekter

OBJEKTORIENTERET PROGRAMMERING

Et objekt

- modsvarer ting og begreber i den virkelige verden
- kan modellere ting fra den virkelige verden
- kan opfattes som en agent, der agerer med andre objekter (agenter)
- har en tilstand bestemt ud fra data, som også kaldes attributter
- kan udføre handlinger, som kaldes metoder
- indkapsler sine data og sine metoder
- er konstrueret ud fra en skabelon (= en klasse)

En klasse

- er en samling af samme slags objekter
- definerer de tilhørende objekters typer

OBJEKTORIENTERET PROGRAMMERING

Et objekt

- modsvarer ting og begreber i den virkelige verden
- kan modellere ting fra den virkelige verden
- kan opfattes som en agent, der agerer med andre objekter (agenter)
- har en tilstand bestemt ud fra data, som også kaldes attributter
- kan udføre handlinger, som kaldes metoder
- indkapsler sine data og sine metoder
- er konstrueret ud fra en skabelon (= en klasse)

En klasse

- er en samling af samme slags objekter
- definerer de tilhørende objekters typer
- er en skabelon til beskrivelse af objekter

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

```
objektnavn = Klassenavn(parametre0)  
  
objektnavn.metode1(parametre1)  
  
objektnavn.metode2(parametre2)  
  
objektnavn.metode3(parametre3)
```

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

```
objektnavn = Klassenavn(parametre0)  
  
objektnavn.metode1(parametre1)  
  
objektnavn.metode2(parametre2)  
  
objektnavn.metode3(parametre3)
```

Klassenavn skrives med stort forbogstav
Navnet på objektet skrives med lille forbogstav

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

```
objektnavn = Klassenavn(parametre0)  
  
objektnavn.metode1(parametre1)  
  
objektnavn.metode2(parametre2)  
  
objektnavn.metode3(parametre3)
```

Klassenavn skrives med stort forbogstav
Navnet på objektet skrives med lille forbogstav

Eksempel:

```
class Bil():  
    def __init__(self, navn):  
        self.navn = navn  
  
bil = Bil('Audi')
```

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

```
objektnavn = Klassenavn(parametre0)  
  
objektnavn.metode1(parametre1)  
  
objektnavn.metode2(parametre2)  
  
objektnavn.metode3(parametre3)
```

Klassenavn skrives med stort forbogstav
Navnet på objektet skrives med lille forbogstav

Eksempel:

```
class Bil():  
    def __init__(self, navn):  
        self.navn = navn  
  
bil = Bil('Audi')
```

bil er et objekt (en instans) af klassen **Bil**

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

```
objektnavn = Klassenavn(parametre0)  
  
objektnavn.metode1(parametre1)  
  
objektnavn.metode2(parametre2)  
  
objektnavn.metode3(parametre3)
```

Parameteren **self** skrives kun i definitionen af klassen

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

```
objektnavn = Klassenavn(parametre0)  
  
objektnavn.metode1(parametre1)  
  
objektnavn.metode2(parametre2)  
  
objektnavn.metode3(parametre3)
```

Parameteren **self** skrives kun i definitionen af klassen

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

```
objektnavn = Klassenavn(parametre0)  
  
objektnavn.metode1(parametre1)  
  
objektnavn.metode2(parametre2)  
  
objektnavn.metode3(parametre3)
```

Parameteren **self** skrives kun i definitionen af klassen

I princippet kan man skrive alt muligt andet end **self**, men det kan ikke anbefales

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

```
objektnavn = Klassenavn(parametre0)  
  
objektnavn.metode1(parametre1)  
  
objektnavn.metode2(parametre2)  
  
objektnavn.metode3(parametre3)
```

Parameteren **self** skrives kun i definitionen af klassen

I princippet kan man skrive alt muligt andet end **self**, men det kan ikke anbefales

En typisk fejl er, at man glemmer at skrive **self** alle de nødvendige steder i klassen

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

```
objektnavn = Klassenavn(parametre0)  
  
objektnavn.metode1(parametre1)  
  
objektnavn.metode2(parametre2)  
  
objektnavn.metode3(parametre3)
```

Alt i Python er objekter og klasser

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

```
objektnavn = Klassenavn(parametre0)  
  
objektnavn.metode1(parametre1)  
  
objektnavn.metode2(parametre2)  
  
objektnavn.metode3(parametre3)
```

Alt i Python er objekter og klasser

Eksempel lister

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

```
objektnavn = Klassenavn(parametre0)  
  
objektnavn.metode1(parametre1)  
  
objektnavn.metode2(parametre2)  
  
objektnavn.metode3(parametre3)
```

Alt i Python er objekter og klasser

Eksempel lister

```
venner = ['Mads', 'Peter', 'Eva', 'Signe']  
venner.append('Tea')
```

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

```
objektnavn = Klassenavn(parametre0)  
  
objektnavn.metode1(parametre1)  
  
objektnavn.metode2(parametre2)  
  
objektnavn.metode3(parametre3)
```

Alt i Python er objekter og klasser

Eksempel lister

```
venner = ['Mads', 'Peter', 'Eva', 'Signe']  
venner.append('Tea')
```

venner bliver ved tildelingen et objekt af klassen Lister

OBJEKTORIENTERET PROGRAMMERING

```
class Klassenavn():  
  
    def __init__(self, parametre0):  
        self.data1 = ...  
        self.data2 = ...  
        ...  
  
    def metode1(self, parametre1):  
        ...  
        ...  
  
    def metode2(self, parametre2):  
        ...  
        ...  
  
    def metode3(self, parametre3):  
        ...  
        ...
```

```
objektnavn = Klassenavn(parametre0)  
  
objektnavn.metode1(parametre1)  
  
objektnavn.metode2(parametre2)  
  
objektnavn.metode3(parametre3)
```

Alt i Python er objekter og klasser

Eksempel lister

```
venner = ['Mads', 'Peter', 'Eva', 'Signe']  
venner.append('Tea')
```

`venner` bliver ved tildelingen et objekt af klassen `Lister` og `venner` bruger en metode `append`, der er defineret i klassen `Lister`

OBJEKTORIENTERET PROGRAMMERING

Eksempel

En kø er en lineær datastruktur, en liste,
med en FIFO-struktur (FirstIn-FirstOut)

OBJEKTORIENTERET PROGRAMMERING

Eksempel

En kø er en lineær datastruktur, en liste,
med en FIFO-struktur (FirstIn-FirstOut)

Operationer på en kø:

- enqueue –tilføj bagerst i køen
- dequeue –fjern den forreste i køen
- isEmpty –undersøg, om køen er tom
- front –bestem den forreste

OBJEKTORIENTERET PROGRAMMERING

Eksempel

En kø er en lineær datastruktur, en liste, med en FIFO-struktur (FirstIn-FirstOut)

Operationer på en kø:

- enqueue –tilføj bagerst i køen
- dequeue –fjern den forreste i køen
- isEmpty –undersøg, om køen er tom
- front –bestem den forreste

```
class Queue:
    def __init__(self):
        self.queue = []

    def isEmpty(self):
        return len(self.queue) == 0

    def enqueue(self, element):
        self.queue.append(element)

    def dequeue(self):
        if not self.isEmpty():
            return self.queue.pop(0)
        else:
            return None

    def front(self):
        if not self.isEmpty():
            return self.queue[0]
        else:
            return None
```

OBJEKTORIENTERET PROGRAMMERING

Eksempel

En kø er en lineær datastruktur, en liste, med en FIFO-struktur (FirstIn-FirstOut)

Operationer på en kø:

- enqueue –tilføj bagerst i køen
- dequeue –fjern den forreste i køen
- isEmpty –undersøg, om køen er tom
- front –bestem den forreste

```
class Queue:
    def __init__(self):
        self.queue = []

    def isEmpty(self):
        return len(self.queue) == 0

    def enqueue(self, element):
        self.queue.append(element)

    def dequeue(self):
        if not self.isEmpty():
            return self.queue.pop(0)
        else:
            return None

    def front(self):
        if not self.isEmpty():
            return self.queue[0]
        else:
            return None
```

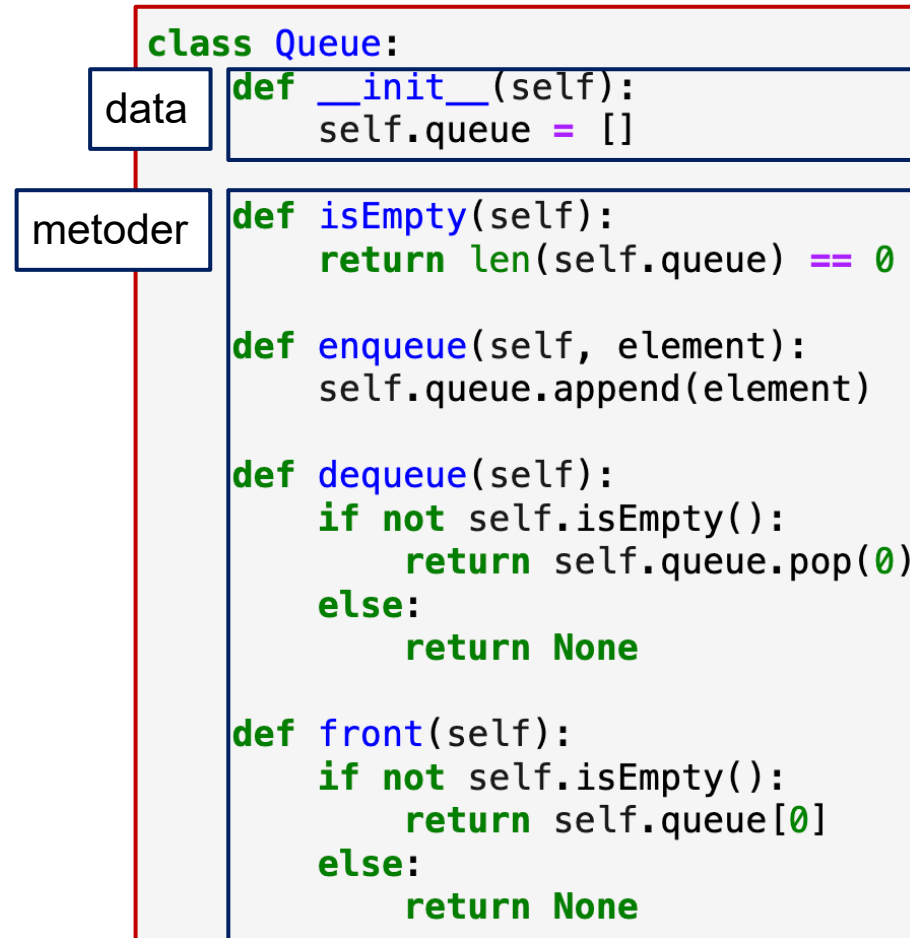
OBJEKTORIENTERET PROGRAMMERING

Eksempel

En kø er en lineær datastruktur, en liste, med en FIFO-struktur (FirstIn-FirstOut)

Operationer på en kø:

- enqueue –tilføj bagerst i køen
- dequeue –fjern den forreste i køen
- isEmpty –undersøg, om køen er tom
- front –bestem den forreste



OBJEKTORIENTETERET PROGRAMMERING

Eksempel

En kø er en lineær datastruktur, en liste, med en FIFO-struktur (FirstIn-FirstOut)

Operationer på en kø:

- enqueue –tilføj bagerst i køen
- dequeue –fjern den forreste i køen
- isEmpty –undersøg, om køen er tom
- front –bestem den forreste

	data
metoder	<pre>class Queue: def __init__(self): self.queue = [] def isEmpty(self): return len(self.queue) == 0 def enqueue(self, element): self.queue.append(element) def dequeue(self): if not self.isEmpty(): return self.queue.pop(0) else: return None def front(self): if not self.isEmpty(): return self.queue[0] else: return None</pre>

```
k1 = Queue()
k1.enqueue(1)
k1.enqueue(2)
k1.enqueue(3)
k1.enqueue(4)
k1.dequeue()
k1.dequeue()
```

OBJEKTORIENTERET PROGRAMMERING

Eksempel

En kø er en lineær datastruktur, en liste, med en FIFO-struktur (FirstIn-FirstOut)

Operationer på en kø:

- enqueue –tilføj bagerst i køen
- dequeue –fjern den forreste i køen
- isEmpty –undersøg, om køen er tom
- front –bestem den forreste

	data
metoder	<pre>class Queue: def __init__(self): self.queue = [] def isEmpty(self): return len(self.queue) == 0 def enqueue(self, element): self.queue.append(element) def dequeue(self): if not self.isEmpty(): return self.queue.pop(0) else: return None def front(self): if not self.isEmpty(): return self.queue[0] else: return None</pre>

```
k1 = Queue()
k1.enqueue(1)
k1.enqueue(2)
k1.enqueue(3)
k1.enqueue(4)
k1.dequeue()
k1.dequeue()
```

```
k1 = [3, 4]
```